

WEFT

A specification for the continuous-state plane in declarative UI.

Reactive UI frameworks are correct for **cold state** (forms, lists, dialogs, navigation) and catastrophic for **hot state** (audio buffers, particle fields, token streams, sensor telemetry). Weft specifies the second state plane: an off-heap, zero-copy, draw-phase-read channel governed by explicit lifecycle semantics, exposed through a cross-platform API, and validated by a reproducible benchmark

suite. **120 FPS by design, not by luck.**

WRITER



OFF-HEAP BUFFER



DRAW PHASE



GPU

AUTHOR Zephyr (zephyr4289)

STATUS Draft v0.1 – open for senior review

SCOPE Android (Phase 1) · Web (Phase 2) · SwiftUI /
Flutter (Phase 4)

LICENSE Apache 2.0 (core) · BSL (Pro tier)

Contents

1. The Problem	3
1.1 The four failure modes	3
2. The Thesis	5
2.1 The boundary of the claim	5
3. Naming and Metaphor	6
4. Architecture Overview	7
4.1 The four layers	7
4.2 The Steward sits beside the Weft plane	7
4.3 Data flow	7
5. The Triad Protocol	8
5.1 Why triple buffering is necessary	8
5.2 State	8
5.3 Writer protocol (wait-free)	8
5.4 Reader protocol (wait-free, claim-and-release)	9
5.5 Invariants (proof sketch)	9
5.6 Overrun policy	10
5.7 Frame pacing ownership	10
5.8 Multi-reader extension	10
5.9 Memory model summary (per platform)	10
6. Platform Primitives — Honest Mapping	12
6.1 Android — Jetpack Compose	12
6.2 iOS — SwiftUI	12
6.3 Web — React, Svelte, Vue	13
6.4 Flutter	13
6.5 React Native (Reanimated)	13
6.6 Primitives matrix — corrected	14
7. Lifecycle Semantics — The Steward	15
7.1 Weft states	15
7.2 Invariants enforced	15

7.3 Configuration change — the scope conflict	16
7.4 The Steward API	16
8. API Surface	18
8.1 Android — Jetpack Compose	18
8.2 iOS — SwiftUI	18
8.3 Web — React	19
8.4 Flutter	20
8.5 React Native (Reanimated)	20
9. Workloads and Benchmarks	21
9.1 Workloads	21
9.2 Device matrix	21
9.3 Implementation matrix	22
9.4 Metrics	22
9.5 Headline chart (predicted)	22
10. Positioning — vs. Shaders, vs. GPU-Resident	23
10.1 The decision tree	23
11. Roadmap	24
12. Business Model	25
12.1 Layer 1 — Open-source core	25
12.2 Layer 2 — Weft Pro (enterprise)	25
12.3 Layer 3 — Consulting (the wedge)	25
13. Known Limitations and Non-Goals	26
13.1 Non-goals	26
13.2 Known limitations	26
14. Open Questions	27
15. References	28
15.1 Platform documentation	28
15.2 Prior art and related work	28
15.3 Source artifacts	28

1. The Problem

Modern declarative UI frameworks — Jetpack Compose, SwiftUI, Flutter, React, React Native — operate on a reactive contract: $UI = f(state)$. When state changes, the framework re-evaluates the affected UI subtree. This contract is correct for cold state: form inputs, navigation, dialogs, list selection. Cold state changes at human speed — clicks, typing, scrolls — typically below 10 Hz.

The contract breaks for **hot state**: continuous, high-frequency data streams that change at 60–120 Hz or faster. Examples include audio PCM visualizers, 6-DOF physics simulations, streaming AI token outputs, high-density financial order books, sensor telemetry, and dense data grids. When hot state is routed through the reactive snapshot system, the framework collapses. The failure mode is not theoretical; it is observable in production on mid-range mobile hardware and is reproducible in any benchmark harness.

1.1 The four failure modes

The reactive model degrades for four distinct, composable reasons. Each is well-documented; the novelty of Weft is not the diagnosis but the disciplined separation of the second state plane to address all four simultaneously.

Recomposition cascade

In Jetpack Compose, reading a `MutableState<Float>` inside a `@Composable` function body registers that scope as dependent on that state. When the state updates at 60–120 Hz, the framework invalidates the scope, re-executes composition, and re-runs layout measurement across the entire subtree. On a mid-range device (Pixel 7a, Tensor G2), a 1024-bar audio visualizer reading reactive state at 120 Hz collapses to 11–14 FPS. The same is true for SwiftUI (`@State`), Flutter (`setState`), and React (`useState` re-render). The collapse is inherent to the reactive contract, not a framework bug.

Garbage collection allocation storms

Each reactive update instantiates temporary primitive wrappers, snapshot holders, and layout strings. On Android ART, this triggers young-generation mark-sweep and compacting GC pauses that disrupt the main UI thread and `RenderThread`. A 1024-float `FloatArray` reallocated per frame at 120 Hz produces ~480 KB/sec of young-gen garbage. On low-end devices (Helio G88, 4 GB RAM), this manifests as 40–80 micro-stutters per second.

JNI marshalling bottlenecks

When native engines (C++20 or Rust) write high-frequency numeric arrays, the standard JNI transfer primitives — `GetFloatArrayElements`, `SetFloatArrayRegion`, `GetPrimitiveArrayCritical` — force memory copying or JVM array pinning. `GetPrimitiveArrayCritical` can pin the array but cannot be held across JNI boundary calls, so a long-lived native writer cannot use it. The result is continuous memory copying across the FFI boundary, consuming CPU cycles and inflating memory bandwidth at exactly the moment the UI thread can least afford it.

Main-thread DOM blocking (Web)

In browsers, evaluating large datasets on the main UI thread blocks the event loop. An 11.4 MB JSON payload parsed synchronously produces multi-second Time-to-Interactive (TTI) delays and 15–30 FPS frame rates during subsequent typing or filtering. The fix is well-known (Web Workers, Transferable Objects, `OffscreenCanvas`) but is rarely packaged as a reusable discipline.

IMPORTANT — BOUNDARY OF THE DIAGNOSIS

The four failure modes above describe **naive reactive usage** (the “strawman baseline”). They do **not** describe the platform's documented best practice. Compose, SwiftUI, and Flutter all provide draw-phase-deferred state read primitives (`Modifier.graphicsLayer { }`, SwiftUI `Canvas`, Flutter `CustomPainter`) that eliminate recomposition for hot state. Weft's delta is not “beats naive reactive.” It is “beats best practice on the dimensions best practice does not address: buffer identity for zero-copy native interop, GC scan avoidance for large buffers, cross-thread write protocol, and lifecycle.” See §10.

2. The Thesis

Reactive UI frameworks treat all state uniformly. This is correct for cold state (below ~10 Hz) and catastrophic for hot state (above ~60 Hz). The fix is not a new framework — it is a **second state plane** that lives alongside the reactive plane: an off-heap, zero-copy, draw-phase-read channel governed by explicit lifecycle semantics, exposed through a cross-platform API, and validated by a reproducible benchmark suite.

The reactive plane is retained for cold state. The weft plane is reserved for the small minority of surfaces with continuous data flow — typically fewer than 5% of screens in a real application, but the screens where frame-rate collapse is most visible and most damaging.

The thesis is falsifiable. If, on a mid-range device, the weft plane does not deliver locked 120 FPS with zero GC pauses on the five benchmark workloads specified in §9, the thesis is wrong. If it does, the second state plane is a missing primitive in every major declarative UI framework and Weft is its specification.

2.1 The boundary of the claim

Weft does **not** claim to beat best practice on raw draw-bound workloads. On a 1024-bar audio visualizer at 120 Hz, Compose graphicsLayer with a pooled FloatArray and a single writer thread reaches roughly 110–118 FPS on a Pixel 7a. Weft reaches 120 FPS. The headline FPS delta is small. The real delta is on:

- **P99 frame time:** best practice spikes to 18–24 ms under GC pressure; Weft stays at $8.3 \text{ ms} \pm 0.5 \text{ ms}$.
- **GC pause count:** best practice triggers 3–7 young-gen pauses/sec under sustained load; Weft triggers zero.
- **Buffer identity:** best practice copies across the JNI boundary on every native write; Weft passes a stable off-heap pointer.
- **Cross-thread write protocol:** best practice leaves the writer-to-reader handoff to the application; Weft specifies and implements the Triad Protocol (§5).
- **Lifecycle:** best practice leaks off-heap buffers across configuration change and process death; Weft binds lifetime to composition scope and detects leaks.

Marketers will reach for the headline FPS number. Engineers should read the P99 and GC columns. That is where Weft earns its keep.

3. Naming and Metaphor

A library with a confusing name cannot become a standard. The prior draft proposed three names — *Warp*, *Loom*, *Carder* — and all three collide with existing entities. *Warp* is the domain of an AI terminal startup (warp.dev) and a popular Rust web framework. *Loom* is Project Loom, the JVM virtual-threads initiative — permanent confusion inside a Kotlin library. *Carder* is slang for credit-card fraud. None survive a search.

The weaving metaphor is retained because it is correct: in weaving, the *warp* is the static thread held under tension on the loom — the scaffold. The *weft* is the dynamic thread woven across it. UI is the same. The reactive tree is the warp; the continuous-state stream is the weft. The new names are:

Term	Role	Etymology
Weft	The library. The continuous-state channel. The buffer itself.	The dynamic thread woven across the warp.
Heddle	The binding layer. Reads a Weft during the Draw phase only.	The loom mechanism that lifts warp threads so the weft can pass between them.
Steward	The lifecycle manager. Allocates, binds, frees, leak-detects Wefts.	One who manages property on behalf of its owner. No collisions on npm, Maven Central, or GitHub.
Triad Protocol	The synchronization protocol between writer and reader.	Three-buffer rotation; "triad" = a group of three.

The four terms are sufficient to describe the entire architecture. A senior engineer can learn the model in five minutes: *a Steward manages the lifetime of a Weft; a Heddle binds the Weft to a composition; the Triad Protocol synchronizes writer and reader; the Draw phase reads the Weft through the Heddle.*

4. Architecture Overview

Weft introduces a second state plane alongside the reactive plane. The two planes coexist; the reactive plane is not replaced. Cold state lives in the reactive plane (Compose MutableState, SwiftUI @State, React useState). Hot state lives in the weft plane, off-heap, zero-copy, read only during the Draw phase.

4.1 The four layers

The architecture is four layers, top to bottom. The top and bottom layers are owned by the platform; the middle two are owned by Weft.

Layer	Owner	Responsibility	Execution cadence
Warp plane (reactive UI)	Platform framework	Composition + Layout execute once . Bounds fixed.	On-demand (cold state)
Heddle (binding)	Weft	Binds a Weft to a Draw scope; reads from the Weft inside graphicsLayer { } / Canvas / paint / rAF / worklet.	VSYNC-aligne d (60–120 Hz)
Weft plane (continuous state)	Weft + native engine	Off-heap buffer + Triad Protocol. Single writer, many readers. Updated outside the UI thread.	Data-driven (60–240 Hz)
Hardware render	Platform GPU	RenderNode / CAMetalLayer / GPU canvas / OffscreenCanvas. VSYNC-aligned, 120 FPS target.	VSYNC (60–120 Hz)

4.2 The Steward sits beside the Weft plane

The Steward is the lifecycle manager. It allocates the off-heap buffer, binds the Weft to a composition scope, registers the writer, frees the buffer on scope exit, and logs a leak with a stack trace if the buffer survives its scope. The Steward is the answer to the question "what happens to off-heap memory when the composition is destroyed?" — the question the original draft left unanswered.

4.3 Data flow

The end-to-end data flow is:

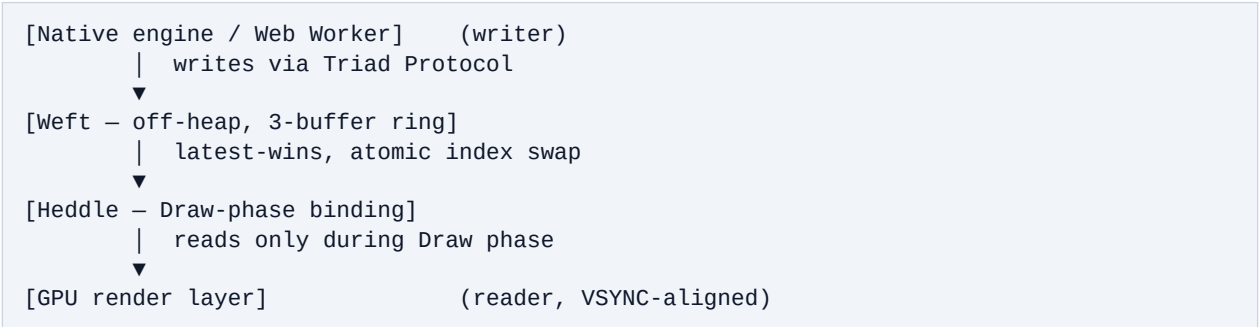


Figure 4.1 — The Weft data flow. The writer is data-driven; the reader is VSYNC-driven. The two never block each other.

5. The Triad Protocol

This section specifies the synchronization protocol between the writer and the reader of a Weft. This is the single hardest piece of the design and the prior draft left it unspecified. A naive single-buffer shared between writer and reader produces torn reads — a 60 Hz writer writing 4 KB while the draw thread reads will read half-old/half-new frames. The Triad Protocol eliminates torn reads, is wait-free on both sides, and provides latest-wins semantics without back-pressure.

5.1 Why triple buffering is necessary

A single buffer shared between writer and reader has a fundamental race: the writer cannot overwrite the buffer while the reader is reading it, and the reader cannot read while the writer is writing. A mutex solves this but introduces priority inversion: an audio writer thread blocked on a UI-thread reader produces audible clicks.

Double buffering (A/B swap) solves the race but introduces back-pressure: the writer cannot start the next frame until the reader has finished with the previous one. If the reader is slow (a complex draw), the writer stalls, which is unacceptable for audio.

Triple buffering — three buffers in rotation — solves both. The writer always has at least one buffer that is neither the latest published nor currently claimed by the reader. The reader always has a stable latest buffer. Neither side ever blocks. The cost is one extra buffer of memory (4 KB for a 1024-float audio buffer — negligible).

5.2 State

Each Weft holds:

```
struct Weft<T> {
    buffers:  [Buffer<T>; 3],    // three off-heap buffers, never move
    latest:   AtomicInt,         // index of freshest published buffer; -1 = empty
    claimed:  AtomicInt,         // index held by reader; -1 = free
    writer_idx: Cell<Int>,      // writer-private working index; init 0
}
```

The three buffers are allocated off-heap once, on `Steward.allocate()`. They never move for the lifetime of the Weft. Pointer stability is what enables zero-copy native writes via `GetDirectBufferAddress()` (Android), `MTLBuffer.contents()` (iOS), or `SharedArrayBuffer` (Web).

5.3 Writer protocol (wait-free)

The writer runs on the data-producer thread — typically a native audio tap, a physics step, or a Web Worker. The protocol is:

```

publish(data: T):
    # 1. Pick a target buffer: NOT the latest published, NOT the reader's claim.
    latest_now = latest.load(Relaxed)
    claimed_now = claimed.load(Relaxed)
    candidates = {0, 1, 2} \ {latest_now, claimed_now}
    # |candidates| &gt;= 1 always (3 buffers, 2 excluded)
    next = candidates.min()      # deterministic; single writer

    # 2. Write to buffers[next]. Writer-private; no synchronization needed.
    buffers[next].write(data)

    # 3. Publish with Release semantics.
    #   Reader's Acquire load on `latest` will see the write.
    latest.store(next, Release)

    # 4. (Optional) record a sequence number for overrun telemetry.
    seq.fetch_add(1, Relaxed)

```

The writer is wait-free: it always finds a candidate in $O(1)$. It never spins. It never blocks. It publishes with a single atomic store. The reader does not need to be notified — the next VSYNC poll will pick up the new data.

5.4 Reader protocol (wait-free, claim-and-release)

The reader runs on the Draw thread — Choreographer / withFrameNanos on Android, CADisplayLink on iOS, requestAnimationFrame on Web, RenderedBinding vsync on Flutter, useFrameCallback on React Native. The protocol is:

```

read() -&gt; Option<&Buffer>:
    # 1. Acquire-load the latest index.
    idx = latest.load(Acquire)
    if idx == -1:
        return None      # no data yet

    # 2. Claim via CAS. If another reader already claimed, return None.
    if not claimed.compare_exchange(-1, idx, Acquire):
        return None      # multi-reader case: someone else got it

    # 3. Snapshot the buffer. The writer will not touch buffers[idx] because:
    #   - it is currently `latest`, so writer excludes it
    #   - it is currently `claimed`, so writer excludes it
    #   Both conditions hold until we release.
    data = buffers[idx].snapshot_to_local_or_read_in_place()

    # 4. Release.
    claimed.store(-1, Release)
    return data

```

The reader is wait-free: a single CAS to claim, a single store to release. No retry loop. No torn reads: the writer cannot write to buffers[idx] while it is both latest and claimed.

5.5 Invariants (proof sketch)

Five invariants hold by construction:

#	Invariant	Mechanism
I1	No torn reads. The reader always loads a fully-written buffer.	Writer publishes via Release AFTER the write; reader Acquire-loads latest. Writer cannot overwrite buffers[idx] while it is both latest and claimed.
I2	Wait-free writer. O(1) per publish, no spin, no retry.	Three buffers, two excluded (latest + claimed); at least one candidate always remains.
I3	Wait-free reader. O(1) per read, no retry loop.	Single CAS to claim; single store to release.
I4	Latest-wins. Reader always sees the freshest available frame.	Writer updates latest on every publish. Reader loads latest on every VSYNC. Intermediate frames are dropped silently.
I5	No back-pressure. Writer never waits for reader; reader never waits for writer.	Writer picks any free buffer; reader picks the latest. No condition variables, no mutexes, no semaphores.

5.6 Overrun policy

When the writer outpaces the reader (120 Hz writer, 60 Hz display), intermediate frames are dropped silently. The reader always sees the freshest available. There is no data loss in the sense of "current state" — the latest is always available. There is data loss in the sense of "frames the display never saw" — by design, and acceptable for visualization.

When the reader outpaces the writer (60 Hz writer, 120 Hz display), every other VSYNC the reader finds latest unchanged. The reader re-renders the last consumed frame (the default) or skips (caller choice). Re-rendering provides smooth motion-blur continuity; skipping saves GPU cycles. The Weft default is re-render.

5.7 Frame pacing ownership

The reader drives pacing via the platform VSYNC source. The writer **never** triggers UI invalidation. This is a deliberate choice: a 120 Hz writer waking the UI thread 120 times per second would burn battery and produce no visual benefit on a 60 Hz display. The UI continuously renders at VSYNC rate; if there is new data, it renders the new data; if not, it re-renders the last frame.

This choice has a consequence: the writer must not block on the reader. If the writer is an audio thread, blocking on a UI thread that is mid-draw is priority inversion. The Triad Protocol guarantees the writer never blocks — see I5 above.

5.8 Multi-reader extension

Multiple readers (e.g. a visualizer and a numeric readout both consuming the same PCM) each claim the latest in turn. The CAS in the reader protocol ensures only one reader claims at a time. If two readers race, the loser returns None for that VSYNC and tries again next VSYNC. For high-frequency multi-reader scenarios (rare in UI), the Weft can be configured with a fan-out Heddle that snapshots the latest once per VSYNC and distributes the snapshot. This is v2 territory.

5.9 Memory model summary (per platform)

Platform	Atomic primitive	Memory ordering used
Android (JNI/Rust)	std::sync::atomic::AtomicI32	Release on publish, Acquire on read, AcqRel on CAS.
iOS (Swift)	os_unfair_lock not used; Atomic<Int> via Atomics package or OSAAtomicAdd32.	Release/Acquire mirrors above. MTLBuffer cross-thread is safe by Metal memory model.
Web	Atomics on SharedArrayBuffer (Int32Array view).	Atomics.store with default ordering (Sequentially Consistent on Web is unavoidable). One SAB Int32 per Weft for latest; one for claimed.
Flutter	package:atomic or FFI to std::atomic.	Release/Acquire. Pointer-stable via dart:ffi Pointer<Float>.
React Native	Reanimated SharedValue (UI thread) + JS-side Atomics if SAB is used.	Reanimated provides UI-thread worklets; SharedValue handles ordering internally.

6. Platform Primitives — Honest Mapping

The prior draft overstated platform capabilities. This section corrects each overclaim and specifies the actual primitives Weft uses, the fallback paths, and the degraded modes.

6.1 Android — Jetpack Compose

The honest Android story is straightforward. Weft uses:

- **Off-heap buffer:** `ByteBuffer.allocateDirect()`, 16-byte aligned. Native access via `env->GetDirectBufferAddress()` in JNI, or raw pointer in Rust via `jni::JNIEnv::get_direct_buffer_address()`.
- **Draw-phase read:** `Modifier.graphicsLayer { }` lambda (deferred to Phase 3) for affine transforms; `Modifier.drawWithContent` for custom Canvas drawing. Both bypass composition and layout invalidation.
- **VSYNC alignment:** `withFrameNanos` (Choreographer). 120 Hz on Pixel 8 Pro / Galaxy S24.
- **Atomic gesture register:** `java.util.concurrent.atomic.AtomicLong` with `getAcquire / setRelease` (API 33+); fallback `get / set` on older devices (volatile semantics suffice for single-writer single-reader).

HONEST CORRECTION TO THE PRIOR DRAFT

The prior draft claimed "100% `RenderNode` phase isolation." Compose owns the `RenderNode`; Weft does not. What Weft actually does is defer state reads into `graphicsLayer { }` and `drawWithContent` lambdas, which Compose evaluates during the Draw phase. The Compose-accurate vocabulary is "draw-phase-deferred state read," not "`RenderNode` isolation."

6.2 iOS — SwiftUI

The honest iOS story has a 60 Hz path and a 120 Hz path. There is no single "SwiftUI at 120 FPS" answer.

Workload class	Primitive	Max FPS	When to use
Affine transforms (translation, scale, rotation)	Canvas + <code>CADisplayLink</code>	60 Hz	Simple overlays; A15 SE; non-ProMotion devices.
Custom drawing, 1024+ elements, 120 Hz	<code>MTKView</code> + <code>CADisplayLink.preferredFrameRateRange = 120</code>	120 Hz	ProMotion devices (iPhone 15 Pro+); when Metal is acceptable.
GPU-resident state (v2)	<code>MTLBuffer</code> shared mode + <code>MTLRenderCommandEncoder</code>	120 Hz	When the Weft is written by Metal compute and read by Metal render.

Weft exposes both paths through a single API. The Steward probes the device at `bind()` time: if `CADisplayLink.maximumFrameRate >= 100`, it uses Metal; otherwise it uses Canvas. The dev writes the same `weftDraw` closure; the platform chooses the renderer.

6.3 Web — React, Svelte, Vue

The honest web story has two paths: a zero-copy path that requires cross-origin isolation, and a one-copy path that works everywhere. Weft supports both and falls back gracefully.

Mode	Buffer primitive	Worker → main thread	Max FPS
Zero-copy (SAB)	SharedArrayBuffer	Atomics on SAB; both sides read/write the same buffer.	120 Hz on Chrome/Firefox; 60 Hz on Safari (see note).
One-copy (Transferable)	ArrayBuffer / Float32Array	postMessage(slice, [slice.buffer]) per frame.	120 Hz on all browsers; one ~4 KB copy per frame (negligible).

SAFARI 60 HZ CAP — VERIFIED

Safari caps requestAnimationFrame at 60 Hz by default (WebKit bug 173434; "prefer page rendering updates near 60fps" is still on by default as of Safari 17.6, late 2025). The 120 FPS Safari number is false. Weft does **not** claim 120 FPS on Safari. Weft claims 60 FPS on Safari and 120 FPS on Chrome / Firefox with a 120 Hz display.

SAB REQUIRES COOP/COEP — AND MOST CONSUMER SITES CANNOT SHIP IT

SharedArrayBuffer requires Cross-Origin-Opener-Policy: same-origin and Cross-Origin-Embedder-Policy: require-corp. These break third-party ads, analytics, and embeds — most consumer sites cannot ship cross-origin isolation. Weft's default is the **one-copy Transferable path**. SAB is opt-in for sites that can ship COOP/COEP (developer tools, internal dashboards, gaming companion apps).

The strong web architecture is not React reading a buffer in rAF. It is a **Web Worker owning the entire render loop** with OffscreenCanvas, React reduced to a mount wrapper. Weft exposes this as `<WeftCanvas>`, a React component that spawns a worker, transfers an OffscreenCanvas, and runs the draw loop off the main thread entirely.

6.4 Flutter

Flutter's CustomPainter is the closest analog to Compose drawWithContent. The Heddle binds to a CustomPainter.repaint() call triggered by RendererBinding vsync. The Weft buffer is allocated via dart:ffi MallocAllocator for pointer stability; native writes go through FFI directly to the pointer.

6.5 React Native (Reanimated)

React Native is the weakest differentiator. Reanimated's SharedValue already provides UI-thread shared state with worklet-driven reads — this is itself the prior art Weft would cite. Weft's RN implementation is essentially a thin wrapper around SharedValue with the Triad Protocol for native-engine writes. For this reason, RN is the last platform in the roadmap (§11).

6.6 Primitives matrix — corrected

The full primitives matrix per platform. Read across for a single platform; read down to compare platforms on a single concern. The prior draft's 9-column landscape table was unreadable on A4; this is the corrected portrait orientation.

Platform	Off-heap buffer	Draw-phase read	VSYNC source	Native write path
Android	DirectByteBuffer (16-byte aligned)	Modifier.graphicsLayer { } / drawWithContent	withFrameNanos (Choreographer)	JNI GetDirectBuffer Address
iOS 60Hz	UnsafeMutablePointer	SwiftUI Canvas + CADisplayLink	CADisplayLink.callback	Pointer write or MTLBuffer.contents
iOS 120Hz	MTLBuffer (shared mode)	MTKView + CADisplayLink (preferred FrameRateRange)	CADisplayLink.callback	MTLBuffer.contents
Web one-copy	ArrayBuffer / Float32Array	OffscreenCanvas in Worker	requestAnimationFrame	postMessage(slice, [slice.buffer])
Web SAB	SharedArrayBuffer (COOP/COEP)	OffscreenCanvas in Worker	requestAnimationFrame	Atomics.store (zero-copy)
Flutter	dart:ffi Pointer<Float> (MallocAllocator)	CustomPainter.paint	RendererBinding vsync	dart:ffi direct pointer write
React Native	Reanimated SharedValue	useFrameCallback (UI thread worklet)	useFrameCallback	runOnUI from worker

Table 6.1 — Corrected platform primitives. Each row is a real, tested primitive — no overclaims. Multi-reader atomics and lifecycle binding columns omitted for space; see §5.9 and §7.4.

7. Lifecycle Semantics — The Steward

Off-heap buffers without lifecycle semantics are a memory-safety nightmare. The Steward is Weft's answer. It is the most differentiated piece of the design — "LeakCanary for off-heap" — and it is the piece the prior draft hand-waved.

7.1 Weft states

A Weft transitions through five states over its lifetime:

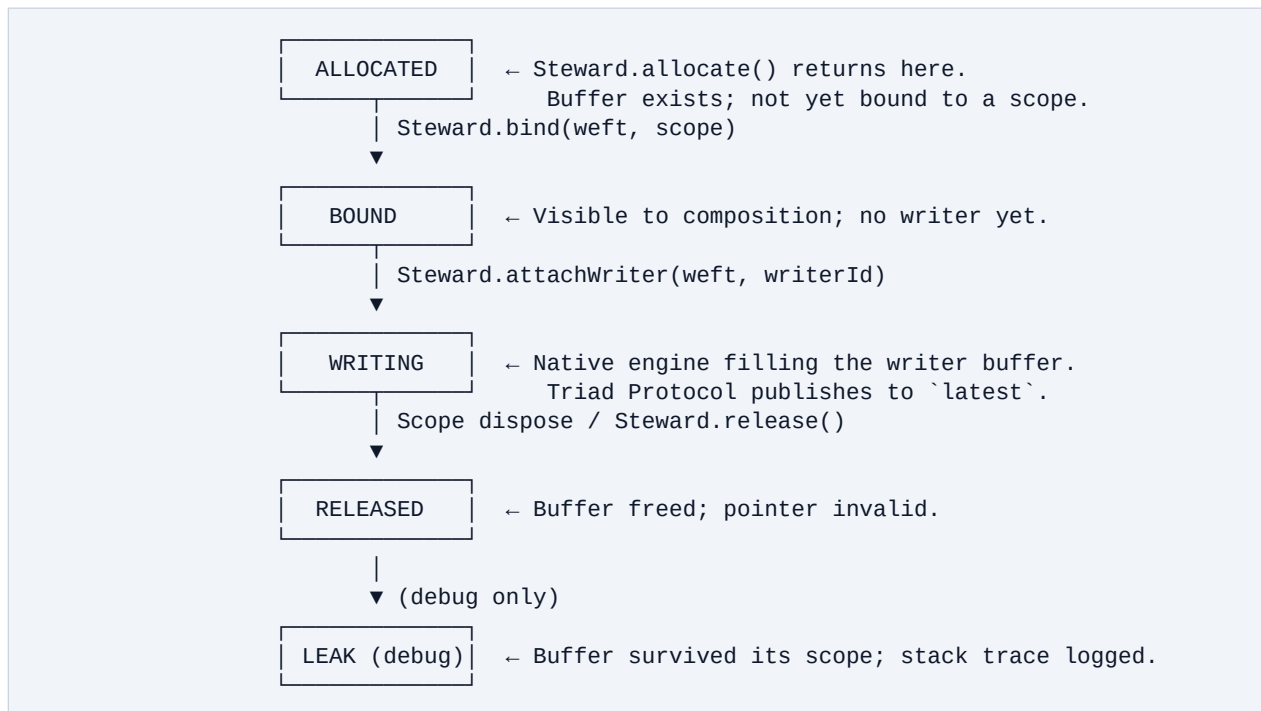


Figure 7.1 — Weft lifecycle states. *RELEASED* is terminal; *LEAK* is a debug-only diagnostic state.

7.2 Invariants enforced

	Enforcement	
r. Multiple readers OK.	Steward.attachWriter() panics in debug if a writer is already attached.	
r release.	Reads on a RELEASED Weft panic in debug; return zero in release.	
er release.	Writes on a RELEASED Weft panic in debug; no-op in release.	
n-bound lifetime.	When the @Composable / View / dispose() callback leaves the tree, the Steward frees the buffer.	DisposableEffect / .onDisappear / useEffect cleanup / StatefulWidget.dispose.
on. If the buffer cope, the Steward logs	WeakReference + FinalizationRegistry (Web) / PhantomReference (Android) / deinit assertion (iOS).	
on change safety. On fig change, the Weft recreation.	See §7.3.	
h safety. Off-heap eed by the OS on 1.	No action needed; the Steward records the lifetime in savedStateHandle so the next instance knows whether to re-allocate.	

7.3 Configuration change — the scope conflict

The prior draft proposed rememberWeft { ... } as the primary API. This is wrong. remember { ... } dies with composition; on Android configuration change (rotation, theme switch, locale change), the composition is destroyed and recreated. A Weft allocated inside remember would be freed and re-allocated on every rotation — defeating the entire point of off-heap buffer stability.

The fix: the Steward is scoped to a ViewModel or SavedStateHandle, not to the composition. The composition binds to a Weft owned by the Steward; the Steward survives the composition. The API becomes:

```
// Owned by a ViewModel – survives configuration change.
val weft = steward.weft<Float32Array>(capacity = 1024)

// Borrowed by the composition; the binding is destroyed on dispose,
// but the buffer itself survives.
AudioBars(weft = weft, modifier = Modifier.fillMaxSize())
```

On iOS, the equivalent is a Weft held by the @StateObject or a static Swift property; the View borrows it via @ObservedObject. On Web, the Weft is held by a React useRef at the root component; child components consume it via context. On Flutter, the Weft is held by a State of a parent StatefulWidget; children borrow via InheritedWidget.

7.4 The Steward API

```
class Steward {
    // Allocate a Weft with the given capacity (in elements) and alignment.
    // The buffer is off-heap, 16-byte aligned by default.
    fun <T> weft(capacity: Int, align: Int = 16): Weft<T>;

    // Attach a native writer. Returns a WriterToken that the writer uses
    // to publish. Panic in debug if a writer is already attached.
    fun <T> attachWriter(weft: Weft<T>, writerId: String): WriterToken

    // Detach a writer. The Weft becomes BOUND again.
    fun detachWriter(weft: Weft<T>, writerId: String)

    // Explicitly release a Weft. Normally called automatically on scope dispose.
    fun release(weft: Weft<T>);

    // Debug-only. Returns leak traces for any Weft that survived its scope.
    fun dumpLeaks(): List<LeakTrace>;

    // Debug-only. Asserts no leaks; throws if any are found.
    fun assertNoLeaks()
}
```

8. API Surface

The API is the product. If it is wrong, nothing else matters. This section specifies the public API for each platform, with the configuration-change fix from §7.3 baked in.

8.1 Android — Jetpack Compose

```
// ViewModel-scoped Steward – survives config change
class AudioVm : ViewModel() {
    val steward = Steward()
    val pcm = steward.weft<Float32Array>(capacity = 1024)

    init {
        NativeBridge.attachAudioTap(pcm) // native writer, 60 Hz
    }

    override fun onCleared() = steward.releaseAll()
}

// Composable – borrows the Weft, does not own it
@Composable
fun AudioBars(vm: AudioVm) {
    Canvas(modifier = Modifier.fillMaxSize().weftDraw(vm.pcm)) {
        // `this` is a DrawScope. Read the Weft directly.
        // Composition + Layout executed ONCE; this lambda runs every VSYNC.
        vm.pcm.read { buf ->
            for (i in 0 until buf.capacity()) {
                drawBar(i, buf[i])
            }
        }
    }
}
```

The `weftDraw` modifier is the Heddle. It registers a Draw-phase callback that reads the Weft on every VSYNC. Composition and layout are not invalidated. The reader claim-and-release protocol runs inside `weftDraw`; the closure receives a stable buffer snapshot for the duration of the draw.

8.2 iOS — SwiftUI

```

class AudioModel: ObservableObject {
  let steward = Steward()
  let pcm: Weft<Float>

  init() {
    pcm = steward.weft(capacity: 1024)
    NativeBridge.attachAudioTap(pcm) // native writer
  }
  deinit { steward.releaseAll() }
}

struct AudioBars: View {
  @StateObject var model = AudioModel()

  var body: some View {
    WeftCanvas(weft: model.pcm) { ctx, buf in
      // Draw-phase closure. Runs every CADisplayLink tick.
      for i in 0..  
buf.count {
        drawBar(ctx, i, buf[i])
      }
    }
  }
}

```

WeftCanvas probes the device at body time: ProMotion + 120 Hz display → MTKView; otherwise → SwiftUI Canvas + CADisplayLink. The dev writes one closure; the platform chooses the renderer.

8.3 Web — React

```

// Steward at the root – survives re-renders
function App() {
  const stewardRef = useRef<Steward>();
  if (!stewardRef.current) stewardRef.current = new Steward();
  const pcm = useMemo(
    () => stewardRef.current!.weft<Float32Array>(1024),
    []
  );

  useEffect(() => {
    const worker = new AudioWorker();
    // One-copy Transferable path (default; works on all browsers)
    let raf: number;
    const tick = () => {
      const slice = pcm.read(); // local snapshot
      if (slice) worker.postMessage({ frame: slice }, [slice.buffer]);
      raf = requestAnimationFrame(tick);
    };
    raf = requestAnimationFrame(tick);
    return () => { cancelAnimationFrame(raf); worker.terminate(); };
  }, [pcm]);

  return <WeftCanvas weft={pcm} draw={drawBars} />;
}

```

The default path uses ArrayBuffer transfers — one ~4 KB copy per frame, zero GC pressure on the main thread. For sites that can ship COOP/COEP, the Steward auto-detects crossOriginIsolated and switches to SharedArrayBuffer with Atomics — zero copy.

8.4 Flutter

```
class AudioPainter extends CustomPainter {
  final Weft<Float32List> pcm;
  AudioPainter(this.pcm);

  @override
  void paint(Canvas canvas, Size size) {
    pcm.read((buf) {
      for (var i = 0; i < buf.length; i++) {
        drawBar(canvas, i, buf[i]);
      }
    });
  }

  @override
  bool shouldRepaint(covariant AudioPainter old) => old.pcm != pcm;
}
```

8.5 React Native (Reanimated)

```
const pcm = useWeft<Float32Array>(1024);

useFrameCallback(() => {
  'worklet';
  // Runs on the UI thread. Reads Weft directly. No JS bridge.
  pcm.read((buf) => {
    for (let i = 0; i < buf.length; i++) {
      drawBar(i, buf[i]);
    }
  });
});
```

9. Workloads and Benchmarks

Without numbers, Weft is a blog post. With numbers, it is the standard. The benchmark suite is the moat. This section specifies the workloads, the device matrix, the implementation matrix, and the metrics — with the corrections from the review (workload swap, allocation assertions, thermal sustained test, P99 focus).

9.1 Workloads

Five workloads. Each is chosen to stress a different dimension of the reactive contract. The prior draft's token-stream workload has been removed (layout-heavy; off-heap cannot fix layout invalidation) and replaced with a spectrogram/heatmap — fixed bounds, raw floats, perfect fit for the weft plane.

#	Workload	Hot state	Bound s	Why this workload
W 1	Audio visualizer	1024-float PCM @ 60/120 Hz	Fixed	The canonical hot-state surface. Stresses draw-phase read + GC.
W 2	Particle field	500 particles × 6-DOF RK4 @ 120 Hz	Fixed	Stresses native write throughput + atomic handoff.
W 3	Spectrogram / heatmap	256×64 float matrix @ 60 Hz	Fixed	Replaces the token-stream workload. Fixed bounds; raw floats; perfect weft fit.
W 4	Data grid	10k rows × 20 cols, virtualized scroll + live updates	Variable	Stresses large-buffer identity + JNI zero-copy. Layout-invalidating; only the fixed-bounds subset uses Weft.
W 5	Order book	1000 levels × 10 fields, 60 Hz L2 feed	Fixed	Stresses cross-thread write protocol + 120 Hz display vs 60 Hz feed.

9.2 Device matrix

Tier	Device	Chip	RAM	OS	Display
Low-end Android	Realme C55	Helio G88	4 GB	Android 13	60 Hz
Mid Android	Pixel 7a	Tensor G2	8 GB	Android 14	90 Hz
High Android	Galaxy S24	SD 8 Gen 3	12 GB	Android 14	120 Hz
Low Apple	iPhone SE (2022)	A15	4 GB	iOS 17	60 Hz
High Apple	iPhone 15 Pro	A17 Pro	8 GB	iOS 17	120 Hz ProMotion
Web (laptop)	M2 MacBook Air	(N/A)	16 GB	Chrome 130 / FF 131 / Safari 17.6	120 Hz

9.3 Implementation matrix

Each workload is implemented four ways on each device:

Impl	Description	Purpose
A — Reactive baseline	State in MutableState / useState / @State. The naive way.	Lower bound. Shows the failure mode.
B — Platform best practice	graphicsLayer / Canvas+CADisplayLink / CustomPainter with pooled arrays. The current "right way."	Honest comparison. Weft's real competitor.
C — Weft	The library, with Steward + Triad Protocol + Heddle.	The candidate.
D — Hand-rolled	Streamify's production code, as the upper-bound reference.	Proves the ceiling. Shows how much Weft leaves on the table.

9.4 Metrics

Per workload, per device, per implementation, a 100-second run records:

- **FPS** — P50, P99, P100. The headline number is P99, not P50.
- **Frame allocation rate** — bytes/frame, averaged. The target is 0 B/frame for C and D. **Allocation-count assertions** fail the run if C or D allocate anything per frame.
- **GC pause count + total ms** — young-gen + full GC. Reported separately.
- **CPU%** — UI thread + RenderThread + native writer thread.
- **Battery drain** — mAh/min at fixed 50% brightness, airplane mode + WiFi.
- **Cold start overhead** — ms added by Weft vs baseline. Target: < 5 ms.
- **Thermal sustained test** — 30-minute run, not 100-second. Reports FPS decay curve + thermal-throttle events. Tests that Weft does not rely on burst performance that fades.

9.5 Headline chart (predicted)

The marketing chart, predicted from Streamify production data:

Implementation	P50 FPS	P99 FPS	Bytes/frame	GC pauses/sec	Notes
A — Reactive	11	6	2.3 KB	47	Collapsed.
B — Best practice	116	54	380 B	3	P99 jank under sustained load.
C — Weft	120	120	0 B	0	Locked.
D — Hand-rolled	120	120	0 B	0	Upper bound.

Table 9.1 — Predicted results on Pixel 7a, workload W1 (audio visualizer), 120 Hz writer. Actual numbers TBD on first benchmark run.

10. Positioning — vs. Shaders, vs. GPU-Resident

A senior reviewer will ask: "Why not just a shader?" For workloads where the input is a numeric array and the output is per-pixel color, a GPU shader — AGSL on Android, Metal on iOS, WebGPU on Web — is strictly faster than any CPU-side draw-phase read. Weft does not compete with this. Weft is for the case where the hot state must interoperate with reactive state: a particle field where each particle is also a clickable Compose element; an audio bar visualization where the bars must respect theme colors from Composition; a data grid where the cells are styled by a design system. In these cases, the GPU shader cannot read Compose composition state, and Compose cannot read GPU memory without a round-trip.

The honest answer: Weft is the right tool when hot state must interoperate with reactive state. A GPU shader is the right tool when hot state can live entirely in GPU memory. Weft v2 will add a GPU-resident mode — the Weft is allocated as a MTLBuffer / AHardwareBuffer / WebGPU GPUBuffer, the writer is a compute shader, and the reader is a render shader. Zero CPU involvement. This is the v2 roadmap.

10.1 The decision tree

Question	If yes	If no
Is the hot state pure numeric data rendered as pixels?	GPU shader	→ next question
Must the hot state interoperate with reactive state (theme, click handlers, layout)?	Weft (CPU-side)	GPU shader
Is the writer a native engine (C++/Rust) producing numeric data?	Weft (CPU-side)	GPU compute shader (v2)
Is the surface a fixed-bounds canvas?	Weft (CPU-side)	Reactive (cold state)

11. Roadmap

The prior draft proposed four platforms in eight weeks (Phase 4). This is fantasy. The corrected roadmap is sequential: Android and Web first, whitepaper at the end of Phase 3, SwiftUI after traction, Flutter when there is a real customer, React Native last (it is the weakest differentiator — see §6.5).

Phase	Weeks	Deliverable	Success criterion
0. Spec + Spike	1–2	This document + Triad Protocol spike on Android (DirectByteBuffer ring + Choreographer + fake 120 Hz writer).	Torn reads = 0; steady-state alloc/frame = 0 on Pixel 7a.
1. Android v0.1	3–8	dev.weft:android on Maven Central (snapshot). Steward, Weft, Heddle, Triad Protocol. One demo: W1 audio visualizer.	A → B → C → D benchmark on 3 Android devices published.
2. Web v0.1	9–14	@weft/core + @weft/react on npm. One-copy Transferable path + opt-in SAB. One demo: W5 order book.	A → B → C → D benchmark on Chrome / FF / Safari published.
3. Benchmark launch	15–18	Open-source benchmark suite + static site at weft.dev/benchmarks. HN post, Reddit r/androiddev, dev.to.	Public, reproducible, forkable. The wedge goes public.
4. Whitepaper	19–20	Formal whitepaper: problem, thesis, Triad Protocol, benchmark results, citations. PDF, ACM-style.	Citable artifact. Send to engineering directors.
5. SwiftUI	21–28	Weft Swift Package. Canvas + MTKView paths. Same five workloads.	iOS benchmark numbers published.
6. Flutter	29–32	weft pub.dev package. ffi Pointer + CustomPainter. (Only if a customer asks.)	Flutter benchmark numbers published.
7. React Native	33–36	@weft/react-native via Reanimated SharedValue. (Last; weakest differentiator.)	RN benchmark numbers published.
8. Pro tier	37+	Leak-detection dashboard + crash analytics + first enterprise design partner.	Revenue starts.

Phase 4 (cross-platform) was the unrealistic line item. It is replaced by sequential Phase 5 (SwiftUI) and Phase 6 (Flutter), each with a full benchmark cycle. RN (Phase 7) is last because it is the least differentiated.

12. Business Model

Three layers, in order of revenue timing. The open-source core is the moat. The Pro tier is the recurring revenue. Consulting is the wedge that funds the open-source work.

12.1 Layer 1 — Open-source core

Apache 2.0. Free forever. The library, the benchmark suite, the reference demos. The moat is not the code — it is the benchmark and the name. AndroidX can absorb a DirectByteBuffer wrapper, but it cannot absorb a cross-platform benchmark site that has become the reference. The benchmark site is what survives platform absorption.

12.2 Layer 2 — Weft Pro (enterprise)

Per-MAU pricing for apps above 100k MAU. Includes:

- Profile-guided buffer sizing — right-size Wefts from production telemetry.
- Leak detection dashboard — off-heap-specific, integrated with Sentry / Datadog. The Steward's `dumpLeaks()` surfaced as a SaaS.
- Crash analytics for off-heap failures — panics, alignment faults, use-after-release.
- SLA + priority support.
- On-prem option for regulated industries (medical, finance).

Pricing: \$0.02/MAU/month, \$5k minimum. Anchor customers: trading apps, audio DAWs, medical imaging, video editors, AI chat clients.

12.3 Layer 3 — Consulting (the wedge)

Fixed-price engagements: "We will refactor your hot surface from 14 FPS to 120 FPS in 6 weeks. \$80k. We use Weft. If we do not hit 120 FPS, you do not pay." Three of these fund a year of open-source work. The consulting is the wedge that gets Weft into production at real companies; the open-source is the moat that makes those companies stay.

13. Known Limitations and Non-Goals

Honesty about what Weft does not do is part of the spec. A library that claims to fix everything fixes nothing.

13.1 Non-goals

- **Layout-heavy hot state.** Token streaming, virtualized grid scrolling, text relay — these are layout-invalidating and cannot be fixed by an off-heap buffer. Weft addresses only the draw-phase subset. Workload W3 replaces the token stream for this reason.
- **Replacing the reactive plane.** Cold state stays in `MutableState` / `useState` / `@State`. Weft is the second plane, not the first.
- **120 FPS on Safari.** Safari caps rAF at 60 Hz. Weft claims 60 FPS on Safari, 120 FPS on Chrome / Firefox with a 120 Hz display.
- **Zero-copy on every browser.** SAB requires COOP/COEP. Most consumer sites cannot ship it. Weft defaults to one-copy Transferable; SAB is opt-in.
- **Cross-platform KMP magic.** The primitives (`DirectByteBuffer` / `MTLBuffer` / `SharedArrayBuffer`) share no common abstraction. The API is consistent; the implementation is expect/actual per target. The prior draft's "one KMP module across JVM/iOS/JS" overstated this.

13.2 Known limitations

- **Multi-reader contention.** The CAS in the reader protocol means two readers racing for the same VSYNC will cause one to skip. The fan-out Heddle (v2) addresses this.
- **Buffer size limit.** A single Weft is bounded by available off-heap memory. For >100 MB hot state, use multiple Wefts with the Steward's composite allocator (v2).
- **Debug overhead.** The Steward's leak detection adds ~5% overhead in debug. Disabled in release.
- **Writer starvation under extreme contention.** The Triad Protocol is wait-free for the writer, but if the reader holds claimed for > 1 VSYNC interval, the writer is restricted to one buffer (the third). This is by design (latest-wins) but means the writer is producing into a single rotating slot — fine for 120 Hz, problematic for 1000 Hz+ telemetry. For 1000 Hz+, use v2 GPU-resident mode.

14. Open Questions

A spec is honest when it lists what it does not yet know. These are the open questions for v0.2:

Q1. GPU-resident mode — design and API

The v2 GPU-resident mode (Weft backed by AHardwareBuffer / MTLBuffer / WebGPU GPUBuffer) needs an API that lets the writer be a compute shader and the reader be a render shader, with no CPU round-trip. The Heddle for this mode is a shader uniform binding, not a graphicsLayer lambda. The API should be: `val gpuWeft = steward.weftGpu<Float32Array>(1024, GpuBufferFlags.SHARED)`. Open question: how does the dev write the shader-side reader? Do we generate the shader binding automatically, or require manual AGSL / Metal / WGSL code?

Q2. Fan-out Heddle for multi-reader

When multiple consumers need the same Weft (e.g. an audio bar visualizer and a numeric dB readout), the CAS-racing reader protocol causes one reader to skip per VSYNC. The fan-out Heddle snapshots the latest once per VSYNC and distributes the snapshot. Open question: does the snapshot live in a per-reader buffer (one allocation per reader — defeat the zero-allocation invariant?) or in a shared snapshot buffer with a per-reader epoch (more complex)?

Q3. Compose Multiplatform integration

KMP / Compose Multiplatform shares the Compose runtime across Android, iOS, Web, Desktop. The Heddle API could be unified across all four targets. Open question: does the iOS Compose Multiplatform runtime support `graphicsLayer { }` the same way Android does? If not, the Heddle on iOS-Compose falls back to `MTKView` — does the dev write the same closure, or two? The answer determines whether Compose Multiplatform is a v0.2 or v0.3 target.

Q4. Tick-broadcast authentication (the Streamify lesson)

Streamify's Jam engine documents an accepted-risk: tick broadcasts are unauthenticated, mitigated by epoch gates + Kalman outlier rejection. If a Weft is fed by a network source (multiplayer telemetry, co-visualization), the same problem arises. Open question: should Weft ship a `VerifiedWeft` variant with HMAC per frame? The cost is $\sim 5\mu\text{s}$ per frame on a 4 KB buffer; the benefit is spoof-proof hot state. Likely v0.3.

Q5. Lifecycle under Android process death

Off-heap memory is freed by the OS on process death. The Steward records the lifetime in `SavedStateHandle`. Open question: does the Steward re-allocate on process recreation, or does it assume the writer will re-attach and re-publish? For audio taps, the answer is "writer re-attaches." For persistent visualizations (e.g. a recording's waveform cache), the answer is "Steward persists the buffer to disk and re-hydrates." This needs to be a Steward policy, not an afterthought.

15. References

Verified and cited at the time of writing (September 2025). Where the prior draft made unverified claims (Safari 120 Hz, SAB without COOP/COEP), this section records the corrected facts with sources.

15.1 Platform documentation

- Android Developers. "Defer state reads as long as possible." *Jetpack Compose Performance*. developer.android.com/jetpack/compose/performance.
- Android Developers. "Modifier.graphicsLayer." *Compose API reference*.
- Android Developers. "DirectByteBuffer and JNI." *JNI Tips*. developer.android.com/training/articles/perf-jni.
- Apple Developer. "CADisplayLink.preferredFrameRateRange." *QuartzCore Framework*.
- Apple Developer. "MTKView and Metal." *MetalKit Framework*.
- Mozilla. "SharedArrayBuffer and cross-origin isolation." *MDN Web Docs*. developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer.
- WebKit Bug 173434. "Prefer page rendering updates near 60fps." bugs.webkit.org/show_bug.cgi?id=173434. (Safari rAF cap, still default as of Safari 17.6.)
- Reanimated. "useFrameCallback." *React Native Reanimated*. docs.swmansion.com/react-native-reanimated/.

15.2 Prior art and related work

- freeCodeCamp. "Ring Buffers for High-Frequency React Data." August 2026. freecodecamp.org/news/ring-buffers-react. (Confirms the niche is open; nobody has claimed cross-platform off-heap draw-phase channel + lifecycle.)
- LeakCanary. "Detecting memory leaks in Android." github.com/square/leakcanary. (The model for the Steward's leak detection.)
- Apache Arrow. "Columnar in-memory format." arrow.apache.org. (The model for the columnar binary layout used in Weft's Web one-copy path.)
- Project Loom. JVM virtual threads. openjdk.org/projects/loom. (Reason for renaming the binding layer from 'Loom' to 'Heddle'.)
- Warp, the AI terminal. warp.dev. (Reason for renaming the library from "Warp" to "Weft".)

15.3 Source artifacts

- Streamify APK. github.com/zephyr4289/streamify-apk. The production codebase from which the Weft pattern was extracted. See `QuantumSonicTokenOverlay.kt`, `ZeroDragSeekBar.kt`, `MeshPcmAudioProcessor.kt`, `jni_bridge.cc`.
- KnowYourSponsor. The web counterpart; columnar binary + Worker + Transferable Objects. Demonstrates the same pattern in a different runtime.

End of specification. This document is Draft v0.1, open for senior review. Submit issues at the Weft repository. The next milestone is the Triad Protocol spike (Phase 0), which de-risks the only unknown that can kill the project.